ML ASSIGNMENT 1

Name: Daksh Srivastava
Roll No.: 150096725087
Cohort: Jeff Bezos

Dataset: train_houses.csv

1. Head

The `head()` function displays the first five rows of the dataset by default. It helps us understand the structure of the dataset and inspect the initial records.

Code: `ass1.head()`

```
ass1.head(10)
```

[5] ✓ 0.0s Python

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	F
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	
5	6	50	RL	85.0	14115	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	N
6	7	20	RL	75.0	10084	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
7	8	60	RL	NaN	10382	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	
8	9	50	RM	51.0	6120	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	
9	10	190	RL	50.0	7420	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	

10 rows × 81 columns

2. Tail

The `tail()` function displays the last five rows of the dataset by default. It helps us inspect the final records.

Code: `ass1.tail()`

```
[6]  ass1.tail(100)
     ✓ 0.0s                                                                    Python
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold
1360	1361	70	RL	51.0	9842	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5
1361	1362	20	RL	124.0	16158	Pave	NaN	IR1	Low	AllPub	...	0	NaN	NaN	NaN	0	6
1362	1363	50	RL	NaN	12513	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	6
1363	1364	60	RL	73.0	8499	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	3
1364	1365	160	FV	30.0	3180	Pave	Pave	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	8
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	MnPrv	NaN	0	2
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	GdPrv	Shed	2500	5
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	4
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	6

100 rows × 81 columns

3. Info

The `info()` function displays the number of entries, column names, non-null counts, data types, and memory usage. It helps identify missing values and understand the structure of the dataset.

Code: `ass1.info()`

```
[8]  ass1.info()
     ✓ 0.0s                                                                    Python
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             1460 non-null   int64
 1   MSSubClass     1460 non-null   int64
 2   MSZoning       1460 non-null   object
 3   LotFrontage    1201 non-null   float64
 4   LotArea        1460 non-null   int64
 5   Street         1460 non-null   object
 6   Alley          91 non-null     object
 7   LotShape       1460 non-null   object
 8   LandContour    1460 non-null   object
 9   Utilities      1460 non-null   object
 10  LotConfig      1460 non-null   object
 11  LandSlope      1460 non-null   object
 12  Neighborhood   1460 non-null   object
 13  Condition1     1460 non-null   object
 14  Condition2     1460 non-null   object
 15  BldgType       1460 non-null   object
 16  HouseStyle     1460 non-null   object
 17  OverallQual    1460 non-null   int64
 18  OverallCond    1460 non-null   int64
 19  YearBuilt      1460 non-null   int64
...
 79  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

4. Describe

The `describe()` function provides descriptive statistics for numerical columns, including count, mean, standard deviation, minimum, maximum, and quartiles.

Code: `ass1.describe()`

```
ass1.describe()
```
[7] ✓ 0.0s Python

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	WoodDeckSF	OpenF
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	1460.000000	...	1460.000000	1460.
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	443.639726	...	94.244521	46.
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456.098091	...	125.338794	66.
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0.000000	...	0.000000	0.
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0.000000	...	0.000000	0.
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383.500000	...	0.000000	25.
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712.250000	...	168.000000	68.
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...	857.000000	547.

8 rows × 38 columns

5. Shape

The `shape` attribute returns the number of rows and columns in the dataset.

Code: `ass1.shape`

```
ass1.shape
```
[4] ✓ 0.0s Python

```
(1460, 81)
```

6. ILOC

The `iloc[]` indexer selects rows and columns using integer positions. It allows us to inspect a specific portion of the dataset.

Code: `ass1.iloc[2:4, 0:5]`

```
ass1.iloc[2:4, 0:5]
```
[9] ✓ 0.0s Python

	Id	MSSubClass	MSZoning	LotFrontage	LotArea
2	3	60	RL	68.0	11250
3	4	70	RL	60.0	9550

7. Condition

Conditional filtering selects records that satisfy a specified condition. This operation can be used to examine houses whose living area exceeds a chosen threshold.

Code: `ass1[ass1["GrLivArea"] > 1000]`

```
ass1[ass1["GrLivArea"] > 1000]
```

[10] ✓ 0.0s Python

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	8
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	MnPrv	NaN	0	2
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	GdPrv	Shed	2500	5
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	4
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	6

1229 rows × 81 columns

8. Line Graph

A line graph represents the relationship between the selected living-area and sale-price values. It helps visualize how these values vary across the records.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 3))
plt.plot(ass1["GrLivArea"], ass1["SalePrice"], linestyle="dotted", marker="o", markersize=5)
plt.title("Living Area vs Sale Price", color="orange")
plt.xlabel("Living Area", color="orange")
plt.ylabel("Sale Price", color="orange")
plt.show()
```

[11] ✓ 0.1s Python



9. Bar Graph

A bar graph displays the number of houses in each overall quality category. It helps compare the frequency of different house-quality ratings.

```python
quality = ass1["OverallQual"].value_counts().sort_index()

plt.figure(figsize=(6, 3))
plt.bar(quality.index, quality.values, color="red", edgecolor="black", width=0.5, alpha=0.5)
plt.title("Number of Houses by Quality", color="darkred", fontweight="bold")
plt.xlabel("Quality", color="darkred")
plt.ylabel("Number of Houses", color="darkred")
plt.show()
```

✓ 0.0s Python



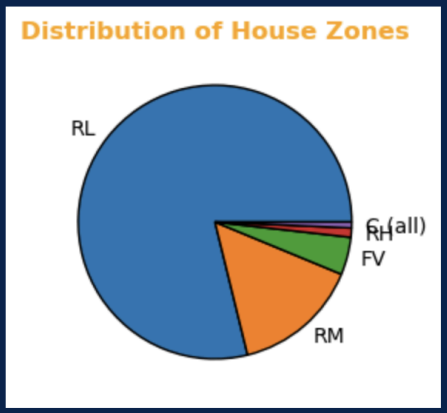
10. Pie Chart

A pie chart represents the distribution of houses across zoning categories. Each slice corresponds to a category and its share of the total.

```python
classes = ass1["MSZoning"].value_counts()

plt.figure(figsize=(6, 3))
plt.title("Distribution of House Zones", color="orange", fontweight="bold")
plt.pie(classes, labels=classes.index, wedgeprops={"edgecolor": "black"})
plt.show()
```

✓ 0.0s

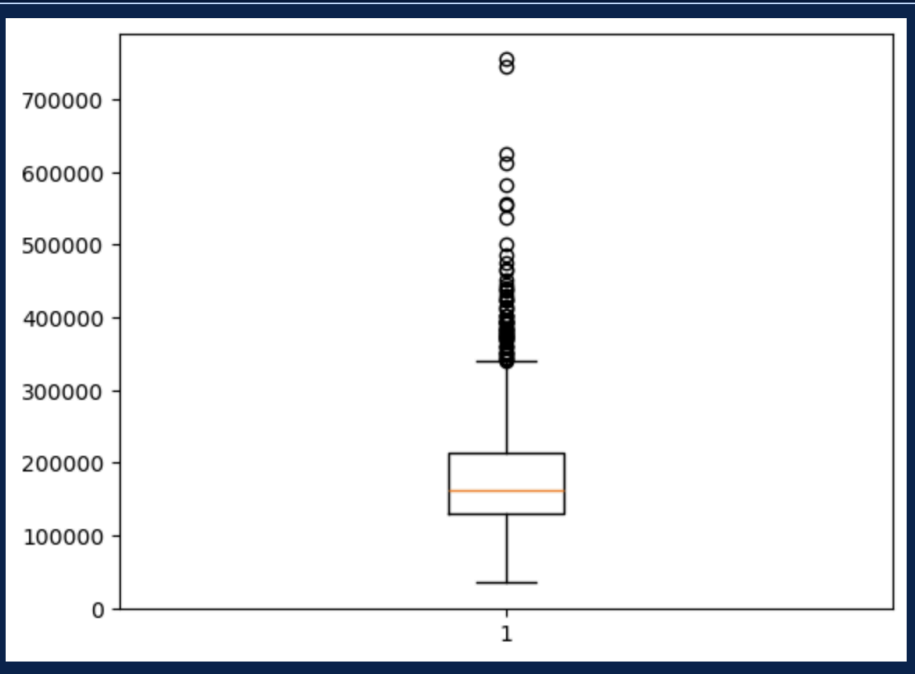


11. Box Plot

A box plot summarizes the distribution of sale prices using the median, quartiles, and whiskers. Points beyond the whiskers may indicate potential outliers.

```
plt.boxplot(ass1["SalePrice"].dropna())
plt.show()
```

[15] ✓ 0.0s

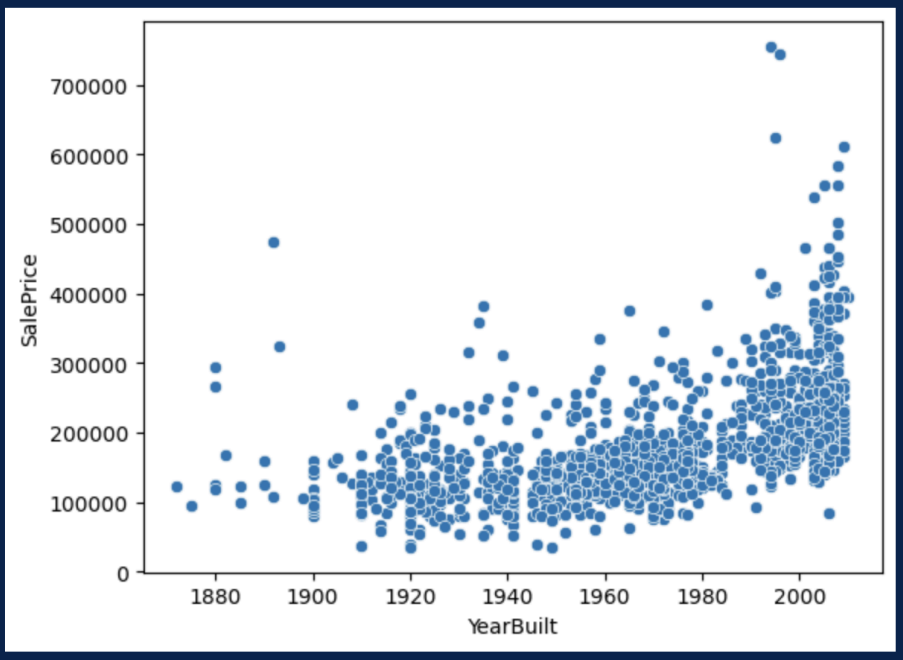


12. Scatter Plot

A scatter plot displays the relationship between the year a house was built and its sale price. Each point represents an individual record.

```
sns.scatterplot(x=ass1["YearBuilt"], y=ass1["SalePrice"])
plt.show()
```

[16] ✓ 0.0s




```
sns.histplot(data=ass1, x="SalePrice", kde=True)
```

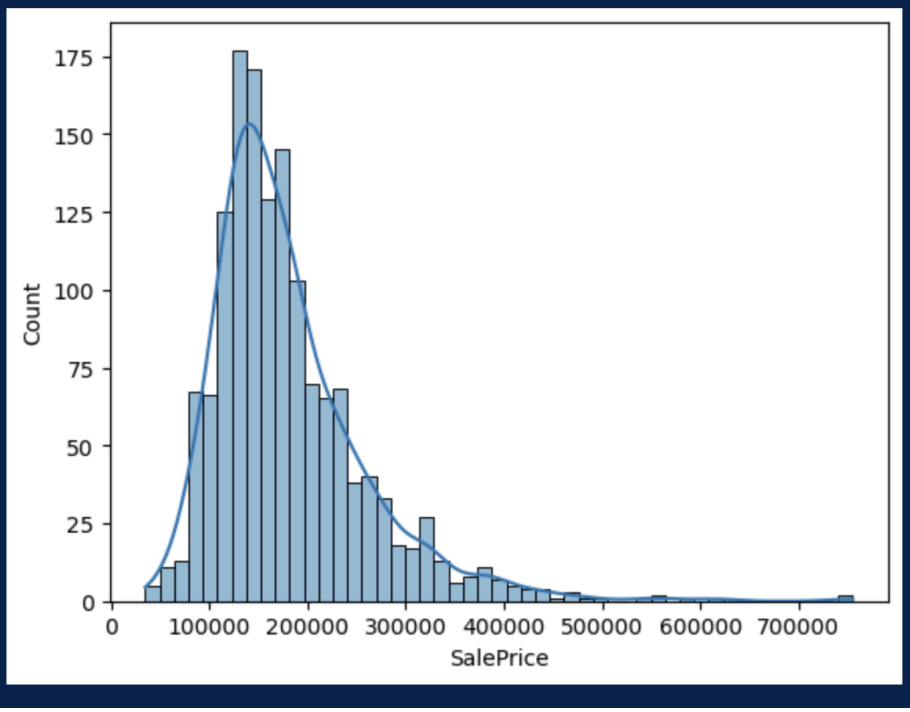
13. Distribution Plot

A distribution plot shows the frequency distribution of sale prices. The histogram displays the distribution of observations, while the KDE curve provides a smoothed estimate of the distrib

```
sns.histplot(data=ass1, x="SalePrice", kde=True)
plt.show()
```

[17] ✓ 0.1s

...



ution.

14. Correlation

Correlation measures the strength and direction of the linear relationship between numerical variables. A correlation matrix helps identify variables that move together.

```
ass1.corr(numeric_only=True)
```

23] ✓ 0.1s Python

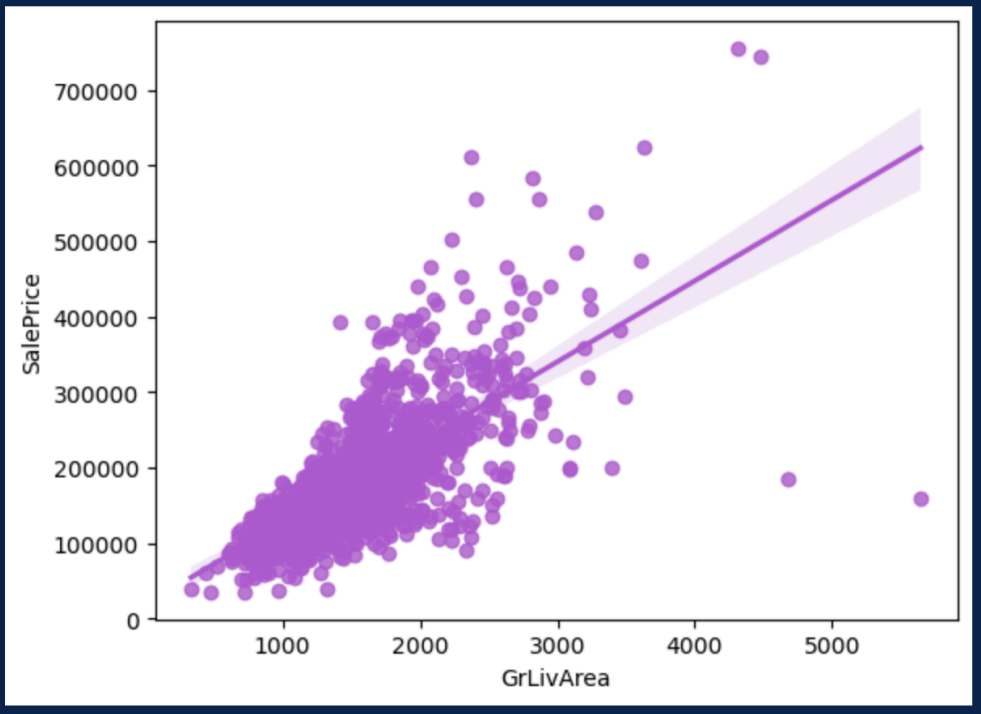
	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	WoodDeckSF	OpenPorch
Id	1.000000	0.011156	-0.010601	-0.033226	-0.028365	0.012609	-0.012713	-0.021998	-0.050298	-0.005024	...	-0.029643	-0.000
MSSubClass	0.011156	1.000000	-0.386347	-0.139781	0.032628	-0.059316	0.027850	0.040581	0.022936	-0.069836	...	-0.012579	-0.006
LotFrontage	-0.010601	-0.386347	1.000000	0.426095	0.251646	-0.059213	0.123349	0.088866	0.193458	0.233633	...	0.088521	0.151
LotArea	-0.033226	-0.139781	0.426095	1.000000	0.105806	-0.005636	0.014228	0.013788	0.104160	0.214103	...	0.171698	0.084
OverallQual	-0.028365	0.032628	0.251646	0.105806	1.000000	-0.091932	0.572323	0.550684	0.411876	0.239666	...	0.238923	0.308
OverallCond	0.012609	-0.059316	-0.059213	-0.005636	-0.091932	1.000000	-0.375983	0.073741	-0.128101	-0.046231	...	-0.003334	-0.032
YearBuilt	-0.012713	0.027850	0.123349	0.014228	0.572323	-0.375983	1.000000	0.592855	0.315707	0.249503	...	0.224880	0.188
YearRemodAdd	-0.021998	0.040581	0.088866	0.013788	0.550684	0.073741	0.592855	1.000000	0.179618	0.128451	...	0.205726	0.226
MasVnrArea	-0.050298	0.022936	0.193458	0.104160	0.411876	-0.128101	0.315707	0.179618	1.000000	0.264736	...	0.159718	0.125
BsmtFinSF1	-0.005024	-0.069836	0.233633	0.214103	0.239666	-0.046231	0.249503	0.128451	0.264736	1.000000	...	0.204306	0.111
BsmtFinSF2	-0.005968	-0.065649	0.049900	0.111170	-0.059119	0.040229	-0.049107	-0.067759	-0.072319	-0.050117	...	0.067898	0.003
BsmtUnfSF	-0.007940	-0.140759	0.132644	-0.002618	0.308159	-0.136841	0.149040	0.181133	0.114442	-0.495251	...	-0.005316	0.129
TotalBsmtSF	-0.015415	-0.238518	0.392075	0.260833	0.537808	-0.171098	0.391452	0.291066	0.363936	0.522396	...	0.232019	0.247
1stFlrSF	0.010496	-0.251758	0.457181	0.299475	0.476224	-0.144203	0.281986	0.240379	0.344501	0.445863	...	0.235459	0.211
2ndFlrSF	0.005590	0.307886	0.080177	0.050986	0.295493	0.028942	0.010308	0.140024	0.174561	-0.137079	...	0.092165	0.208
LowQualFinSF	-0.044230	0.046474	0.038469	0.004779	-0.030429	0.025494	-0.183784	-0.062419	-0.069071	-0.064503	...	-0.025444	0.018
GrLivArea	0.008273	0.074853	0.402797	0.263116	0.593007	-0.079686	0.199010	0.287389	0.390857	0.208171	...	0.247433	0.330
BsmtFullBath	0.002289	0.003491	0.100949	0.158155	0.111098	-0.054942	0.187599	0.119470	0.085310	0.649212	...	0.175315	0.067
BsmtHalfBath	-0.020155	-0.002333	-0.007234	0.048046	-0.040150	0.117821	-0.038162	-0.012337	0.026673	0.067418	...	0.040161	-0.025
FullBath	0.005587	0.131608	0.198769	0.126031	0.550600	-0.194149	0.468271	0.439046	0.276833	0.058543	...	0.187703	0.259
HalfBath	0.006784	0.177354	0.053532	0.014259	0.273458	-0.060769	0.242656	0.183331	0.201444	0.004262	...	0.108080	0.199
BedroomAbvGr	0.037719	-0.023438	0.263170	0.119690	0.101676	0.012980	-0.070651	-0.040581	0.102821	-0.107355	...	0.046854	0.093
KitchenAbvGr	0.002951	0.281721	-0.006069	-0.017784	-0.183882	-0.087001	-0.174800	-0.149598	-0.037610	-0.081007	...	-0.090130	-0.070
TotRmsAbvGrd	0.027239	0.040380	0.352096	0.190015	0.427452	-0.057583	0.095589	0.191740	0.280682	0.044316	...	0.165984	0.234
Fireplaces	-0.019772	-0.045569	0.266639	0.271364	0.396765	-0.023820	0.147716	0.112581	0.249070	0.260011	...	0.200019	0.169
GarageYrBlt	0.000072	0.085072	0.070250	-0.024947	0.547766	-0.324297	0.825667	0.642277	0.252691	0.153484	...	0.224577	0.228
GarageCars	0.016570	-0.040110	0.285691	0.154871	0.600671	-0.185758	0.537850	0.420622	0.364204	0.224054	...	0.226342	0.213
GarageArea	0.017634	-0.098672	0.344997	0.180403	0.562022	-0.151521	0.478954	0.371600	0.373066	0.296970		0.224666	0.241

15. Regression Plot

A regression plot displays the relationship between living area and sale price with a fitted regression line. It helps visualize the overall linear trend in the data.

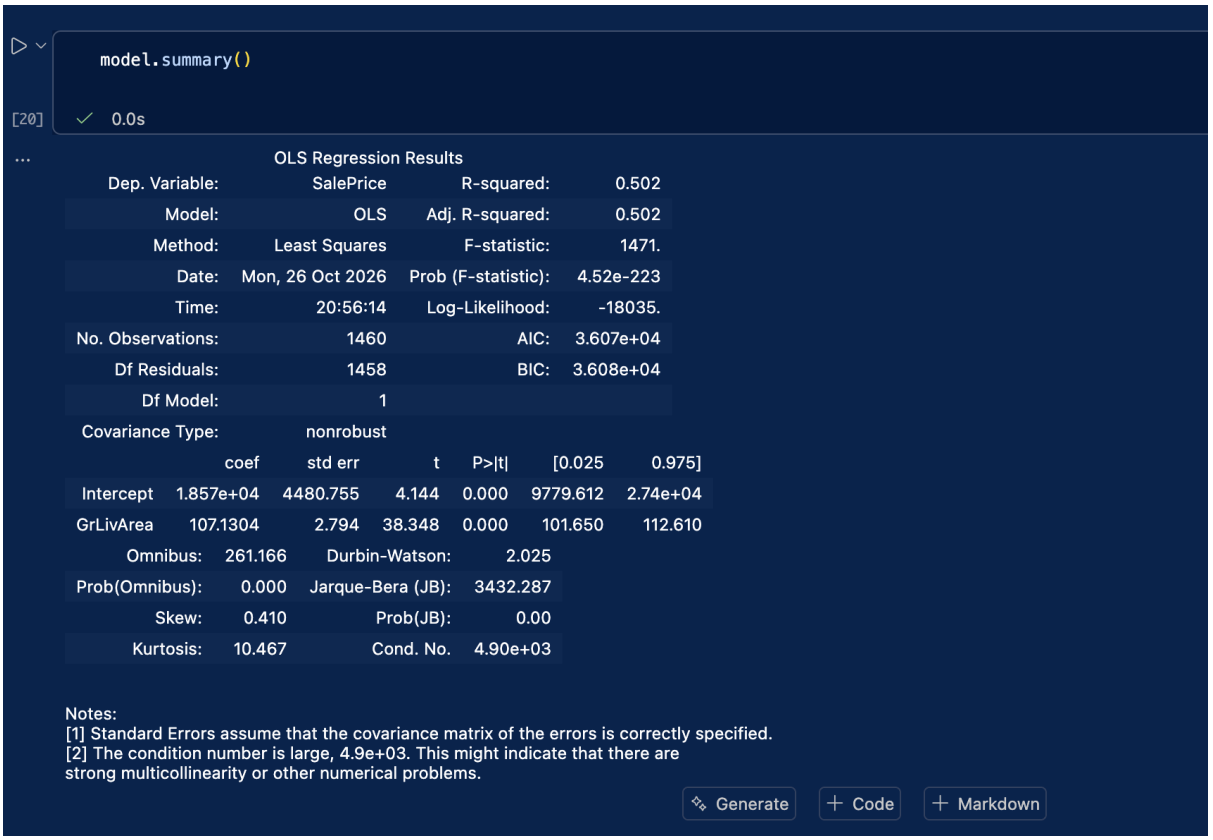
```
sns.regplot(x="GrLivArea", y="SalePrice", data=ass1, color="mediumorchid")
plt.show()
```

[18] ✓ 0.1s



16. Model Summary

The ordinary least squares (OLS) regression model estimates the relationship between sale price and living area. The model summary reports statistics such as R-squared, coefficients, standard errors, t-statistics, and p-values.



```
model.summary()
```

[20] ✓ 0.0s

OLS Regression Results

Dep. Variable:	SalePrice	R-squared:	0.502
Model:	OLS	Adj. R-squared:	0.502
Method:	Least Squares	F-statistic:	1471.
Date:	Mon, 26 Oct 2026	Prob (F-statistic):	4.52e-223
Time:	20:56:14	Log-Likelihood:	-18035.
No. Observations:	1460	AIC:	3.607e+04
Df Residuals:	1458	BIC:	3.608e+04
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.857e+04	4480.755	4.144	0.000	9779.612	2.74e+04
GrLivArea	107.1304	2.794	38.348	0.000	101.650	112.610

Omnibus:	261.166	Durbin-Watson:	2.025
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3432.287
Skew:	0.410	Prob(JB):	0.00
Kurtosis:	10.467	Cond. No.	4.90e+03

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 4.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Generate + Code + Markdown